Programlama Dilleri

Hayri Volkan Agun

Bursa Teknik Üniversitesi

Bilgisayar Mühendisliği Bölümü

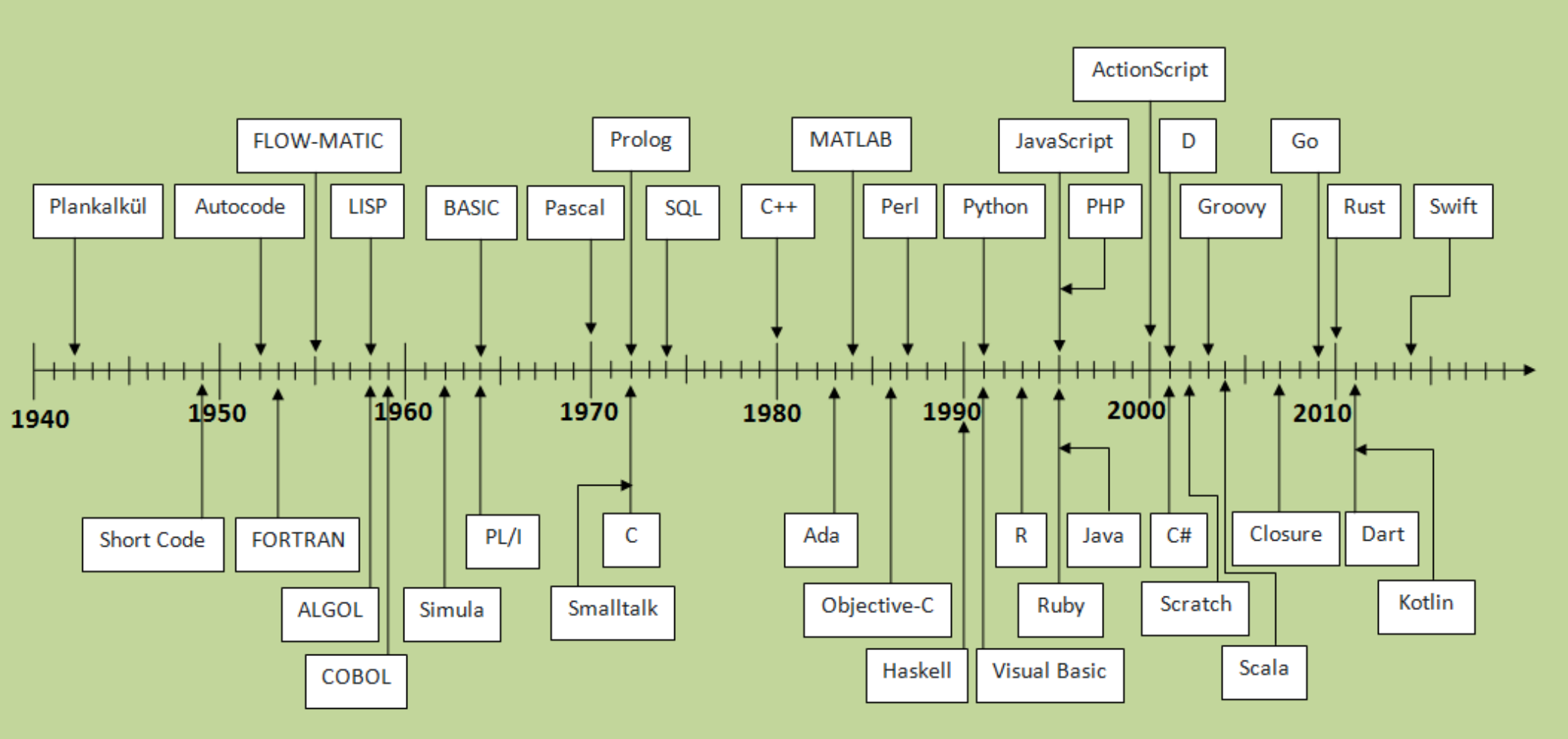
Programlama Dilleri

- Programlama dillerini kavramadan yeni programlama dillerinin geliştirilmesi çok sığ bir yaklaşımdır.
- Bir programlama dilini çok iyi bilmek ve bu dilin yapısını kavramak yenisini geliştirmek için mutlaka gereklidir.
- Bir problemin çözümünde kullanılacak dili seçebilmek için birçok dile hakim olmak bir avantajdır.
- Bir programlama dilinin diğerinden daha çok kullanılması kolay öğrenilir olmasına bağlıdır. Ancak birçok kolay öğrenilen dil düzgünce tasarlanmış iyi bir dil demek değildir.

Programlama Dilleri

- Fortran (1940): Sadece bilimsel hesaplamalar, kayar nokta (floating point) işlemleri için geliştirilmiştir. Halen kullanılmaktadır.
- Algol (1950): Bilimsel hesaplamalar, kayar nokta (floating point) işlemleri için geliştirilmiştir. Fortrandan daha fazla uygulama alanı için kod barındırmaktadır.
- Cobol (1960): Bankacılık gibi işlemlerde sıklıkla kullanılmış yüksek sevyeli bir dildir.
- LISP (1965): Yapay zeka uygulamaları için geliştirilmiş ilk yüksek sevyeli fonksiyonel programlama dilidir.
- Prolog (1965): Yapay zeka uygulamaları için geliştirilmiş ilk yüksek sevyeli sembolik programlama dilidir.
- HTML (1995): XML gibi bir makup dil olarak adlandırılır. Web görüntüleme için geliştirilmiştir.
- Javascript (2000): Web uygulamalarında istemcide browser tarafından çalıştırılan bir dildir.

Programlama Dilleri



Programlama Dilleri

Kriterler	Okunabilirlik	Yazılabilirlik	Güvenilirlik
Basitlik	●	●	●
Ortogonallik	●	●	●
Veri tipleri	●	●	●
Sözdizim tasarımı	●	●	●
Genelleme Yatkınlığı		●	●
İfade edilebilirlik		●	●
Tür denetimi			●
Hata yakalama			●
İsimlendirme kısıtı (r. aliasing)			●

Programlama Dilleri

- 1970’lerden önce program yazmak hız ve etkinlik bakımından değerlendirilirdi.
- Okunabilirlik içinde gösterilen basitlik gibi kavramlar makine kodunda daha fazla öne çıkarken okunabilirlik makine kodu için görece kötüdür.
- Okunabilirlik deyince işlemlerin operandlar ve söz dizim tarafında kolayca anlaşılabilir olması gerekir. Örneğin + ekleme yada toplama şeklinde yorumlanırken – iaretinin listeden çıkarma şeklinde yorumlanması gibi

Programlama Dilleri – Ortogonallik

- Diklik yada ortogonallik tipler üzerinde tanımlanan veri yapıları birbirine diklik şeklinde düşünülebilir. Burada liste içinde int yada float tutyorsa bu yeni bir veri yapısı şeklinde tanımlanmaz. int ve list ayrıca float ve list birbirine diktir.
- Benzer şekilde işaretçiler (pointer) içinde herhangi bir tipi point edebilir. Burada pointer ve basit (primitive (int, vs)) tipler birbirine diktir denir.
- Ortogonal olma, sadelikle yakından ilişkilidir: Bir dilin tasarımı ne kadar ortogonal olursa, dil kuralları o kadar az istisna gerektirir. Daha az istisna, tasarımda daha yüksek düzeyde düzenlilik anlamına gelir; bu da dili öğrenmeyi, okumayı ve anlamayı daha kolay hale getirir.

Programlama Dilleri – Ortogonallik

- Bazılarına göre fonksiyonel diller, sadelik ve ortogonalitenin iyi bir birleşimini sunar. Lisp gibi bir fonksiyonel dilde hesaplamalar, temel olarak fonksiyonların verilen parametrelere uygulanmasıyla gerçekleştirilir. Buna karşılık, C, C++ ve Java gibi emredici (imperative) dillerde hesaplamalar genellikle değişkenler ve atama ifadeleri kullanılarak tanımlanır.
- Fonksiyonel diller, her şeyi diğer fonksiyon çağrılarıyla basitçe birleştirilebilen tek bir yapı — fonksiyon çağrısı — ile gerçekleştirebildikleri için potansiyel olarak en yüksek genel sadeliği sunarlar.

Programlama Dilleri – Veri Tipleri

- Örneğin, bir dilde Boolean (mantıksal) tür bulunmadığı için bir gösterge bayrağı (indicator flag) için sayısal bir tür kullanıldığını varsayalım. Böyle bir dilde şu tür bir atama yapılabilir:

  timeOut = 1

- Bu ifadenin anlamı açık değildir. Oysa Boolean türünü destekleyen bir dilde aynı durum şu şekilde ifade edilir:

  timeOut = true

- Bu ifadenin anlamı ise tamamen açıktır ve yoruma yer bırakmaz.

Programlama Dilleri – Söz dizim tasarımı

- Özel kelimeler: for, while, def gibi özel terimlerin tasarımı. Özel kelimelerin değişken ismi yerine kullanılması problemi (Fortran 95).
- İfadelerin tasarımında, görünümlerinin en azından kısmen amaçlarını yansıtması, okunabilirlik açısından açık bir avantaj sağlar. Anlam (semantik), biçimden (söz dizim) doğrudan çıkarılabilmelidir.
- Bazı durumlarda bu ilke ihlal edilir: Görünüş olarak aynı ya da benzer olan iki dil yapısı, bağlama bağlı olarak farklı anlamlar taşıyabilir.
- Örneğin C dilinde static anahtar sözcüğünün anlamı, kullanıldığı bağlama göre değişir:

Programlama Dilleri – Söz dizim tasarımı

- Eğer static terimi bir fonksiyon içinde tanımlanan bir değişken üzerinde kullanılırsa, bu değişkenin derleme zamanında oluşturulduğu (ve ömrünün program süresince devam ettiği) anlamına gelir.
- Eğer tüm fonksiyonların dışında tanımlanan bir değişken üzerinde kullanılırsa, bu değişkenin yalnızca tanımının bulunduğu dosya içinde görünür olduğu anlamına gelir; yani başka dosyalara dışa aktarılmaz (external linkage’a sahip değildir).
- Bu durum, aynı sözdizimsel biçimin farklı bağlamsal anlamlar taşımasına bir örnektir.

Programlama Dilleri - Yazılabilirlik

- Okunabilirlikte olduğu gibi, yazılabilirlik (writability) de bir dilin hedef problem alanı bağlamında değerlendirilmelidir. Dillerden biri belirli bir uygulama alanı için tasarlanmışken diğeri tasarlanmamışsa, o uygulama alanında bu iki dilin yazılabilirliğini karşılaştırmak adil değildir.
- Örneğin, Visual Basic (VB) ile C'nin yazılabilirlikleri, grafiksel kullanıcı arayüzüne (GUI) sahip bir program geliştirme açısından büyük ölçüde farklıdır; bu tür uygulamalar için VB oldukça uygundur.
- Benzer şekilde, bir işletim sistemi gibi sistem programları yazma konusunda da bu iki dilin yazılabilirlikleri belirgin biçimde farklıdır; çünkü C dili özellikle bu tür sistem yazılımları için tasarlanmıştır.

Programlama Dilleri – İfade edilebilirlik

- Daha yaygın olarak bu kavram, bir dilin hesaplamaları ifade etmek için hantal değil, görece daha kullanışlı yollar sunması anlamına gelir.
- Örneğin C dilinde count++ gösterimi, count = count + 1 ifadesine göre daha kısa ve daha pratiktir.
- Ayrıca Ada dilindeki and then Boolean işleci, bir Boolean ifadesinin **kısa devre (short-circuit) değerlendirmesini** belirtmek için kullanışlı bir yoldur.
- Java dilinde for deyiminin bulunması da sayma (counting) döngülerini yazmayı kolaylaştırır; gerçi aynı iş while ile de yapılabilir.
- Bu tür özelliklerin tümü, bir programlama dilinin yazılabilirliğini artırır.

Programlama Diileri - Güvenilirlik

- Tür denetimi (type checking), bir programdaki tür hatalarının ya derleyici tarafından ya da programın çalışması sırasında tespit edilmesidir. Tür denetimi, bir dilin güvenilirliği açısından önemli bir faktördür.
- Çalışma zamanı (run-time) tür denetimi maliyetli olduğundan, derleme zamanı (compile-time) tür denetimi daha tercih edilir. Ayrıca, programlardaki hatalar ne kadar erken aşamada tespit edilirse, gerekli düzeltmelerin maliyeti de o kadar düşük olur.
- İsimlendirme kısıtı, aliasing bir programda aynı bellek hücresine erişmek için kullanılabilen iki ya da daha fazla farklı ismin bulunmasıdır.
- Günümüzde genel olarak isimendirme kısıtı bir programlama dilinde tehlikeli bir özellik olduğu kabul edilmektedir.

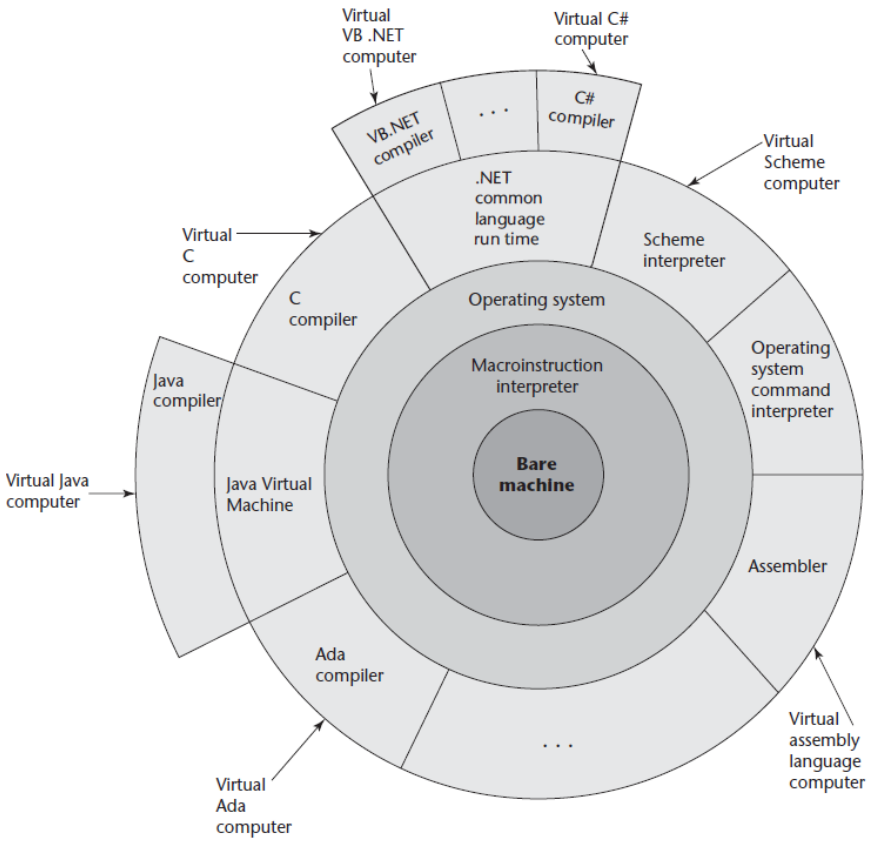
Programlama Dilleri - Maliyet

- Bir programlama dilinin toplam maliyeti birçok özelliğine bağlıdır:
- **Eğitim maliyeti:** Dilin sadeliği ve ortogonalitesi ile programcıların deneyimine bağlıdır. Daha güçlü diller genellikle öğrenmesi daha zor olabilir.
- **Program yazma maliyeti:** Dilin yazılabilirliğine ve hedef uygulama alanına ne kadar uygun olduğuna bağlıdır. Yüksek seviyeli diller, yazılım geliştirme maliyetini düşürmek amacıyla tasarlanmıştır.
- **Derleme maliyeti:** Derleyicilerin verimliliğine bağlıdır. Örneğin, Ada'nın ilk derleyicileri çok pahalıydı, ancak zamanla bu sorun azalmıştır.
- **Çalıştırma maliyeti:** Dilin tasarımına bağlıdır. Çok sayıda çalışma zamanı tür denetimi gerektiren diller hızlı çalışmayı zorlaştırır. Günümüzde yürütme verimliliği, erken dönem dillere kıyasla daha az önceliklidir.

Programın Çalıştırılması

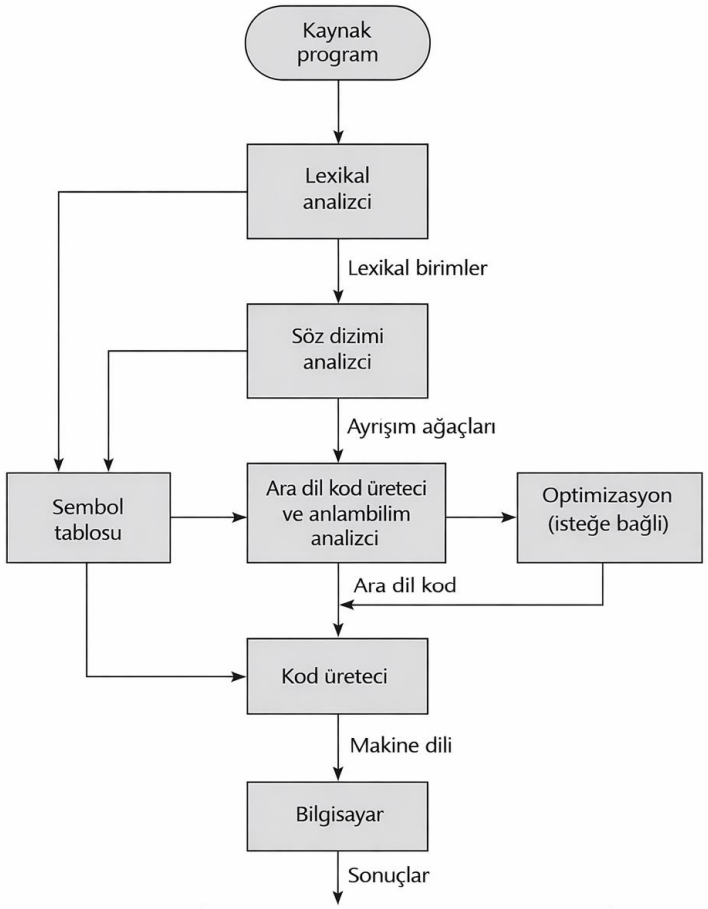
- Programlama dilleri genel olarak üç yöntemden biriyle gerçekleştirilebilir.
- Bir uçta, programlar makine diline çevrilir ve bilgisayarda doğrudan çalıştırılır. Bu yönteme **derleyici (compiler) gerçekleştirim** denir. Çeviri işlemi tamamlandıktan sonra programlar çok hızlı çalışır. C, COBOL ve C++ gibi dillerin çoğu üretim düzeyindeki gerçekleştirimleri derleyiciler aracılığıyladır.

Programın Çalıştırılması



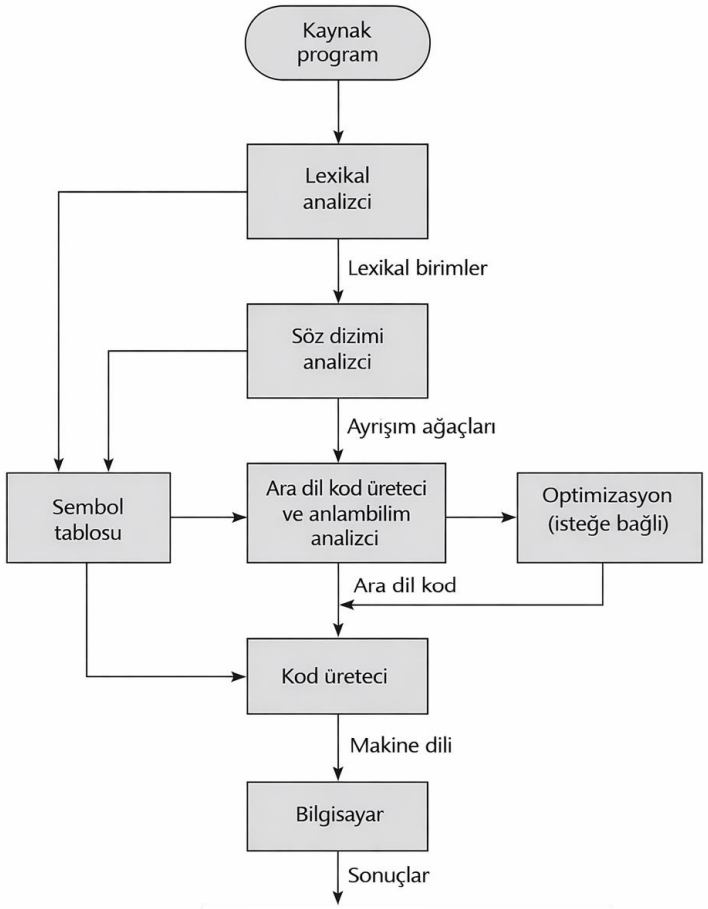
- Direk derleme yoluyla ikili koda çevrilir.
- Ara bir diler çevrilir ve bu dil yorumlayıcı şeklinde anlık olarak kod üreterek programı çalıştırır. Java ve C# .NET örneği.

Derleme



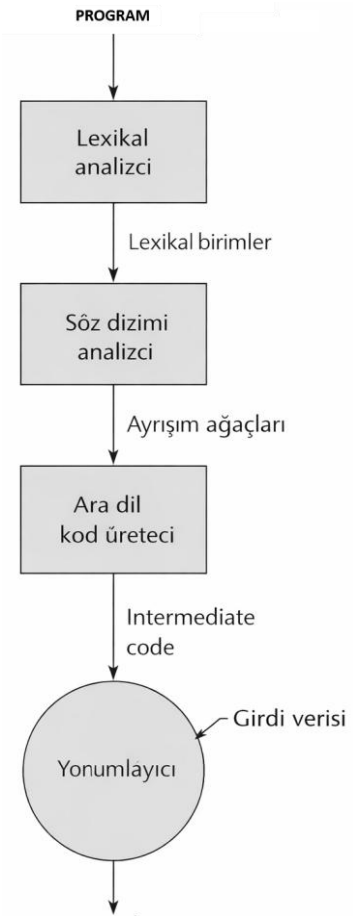
- Derleme işleminde kod önce kelime ve terim analizinden geçer daha sonra söz dizim analizi ile ya ara kod üretilir yada iki koda dönüşecek assembly kodu üretilir.
- Ara kod üretiminde optimizasyon yapılabilir sonuçta elde edilen ikili kod çalıştırılır.

Derleme



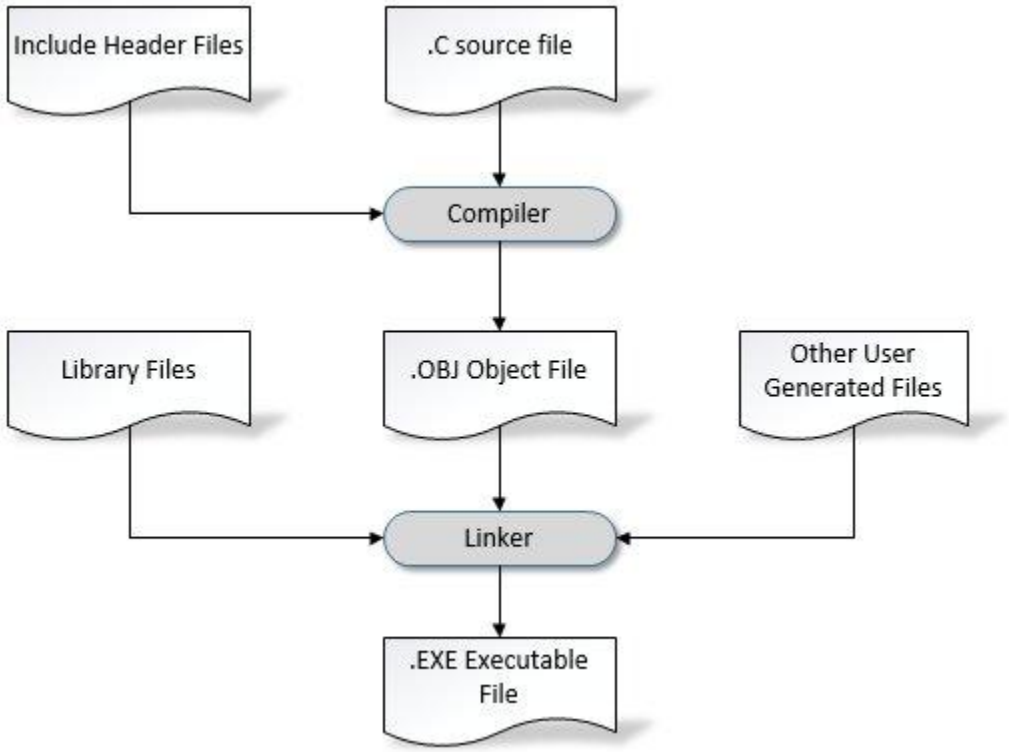
- Bazı dil gerçekleştirim sistemleri, derleyiciler ile saf yorumlayıcılar arasında bir uzlaşma niteliğindedir; bu sistemler yüksek seviyeli programları kolay yorumlanabilir bir ara dile çevirir ve kaynak kod yalnızca bir kez çözümlendiği için saf yorumlamaya göre daha hızlıdır; bu tür sistemlere hibrit gerçekleştirim sistemleri denir ve ara kod makine diline çevrilmeden doğrudan yorumlanır.
- Perl hibrit bir sistemle gerçekleştirilmiş olup yorumlama öncesinde kısmi derleme yapılır; Java'nın ilk sürümleri de hibritti ve ara biçimi olan bytecode, bytecode yorumlayıcısı ile çalışma zamanı sistemi bulunan makinelerde taşınabilirlik sağlayarak Java Virtual Machine (JVM) yapısını oluşturur; günümüzde Java bytecode'unu doğrudan makine koduna çeviren sistemler de vardır.

Hibrit Derleme



- Just-in-Time (JIT) sistemlerde ise program önce ara dile çevrilir, çalıştırma sırasında çağrılan yöntemler makine koduna derlenir ve sonraki çağrılar için saklanır; JIT, Java ve .NET dilleri için yaygın olarak kullanılmaktadır.

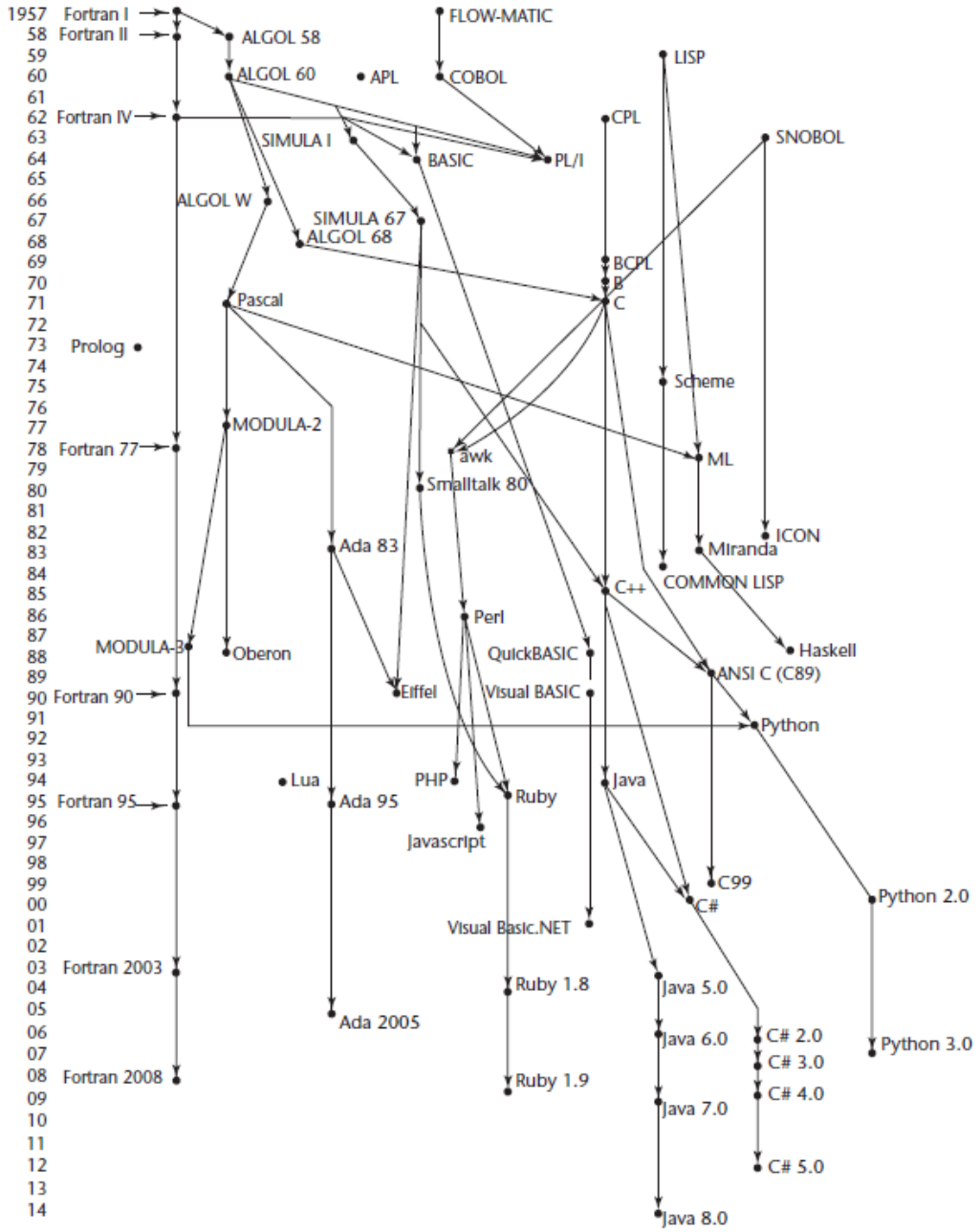
Linker



- Derleyici tarafından üretilen kodların ve farklı kütüphanelerin bir araya getirlmesinden Linker sorumludur.
- Linker programa nihai halini verir.
- Yanda C programlama diline ait bir kod bloğunun çalıştırılabilir exe dosyasının oluşturulmasında Linker’ın görevi gösterilmektedir.

Dillerin etkileşimi

- Lisp çeyrek yüzyıl boyunca yapay zekâ uygulamalarına tamamen hâkim olmuştur. Lisp'in son derece verimsiz olduğu yönündeki ününün büyük kısmı ortadan kalkmıştır. Günümüzdeki birçok gerçekleştirim derlenmektedir ve ortaya çıkan kod, kaynak kodun bir yorumlayıcı üzerinde çalıştırılmasına kıyasla çok daha hızlıdır.
- Lisp, yapay zekâdaki başarısının yanı sıra, programlama dillerinde canlı bir araştırma alanı olduğu kanıtlanan fonksiyonel programlamanın da öncüsüdür. Bölüm 1'de belirtildiği gibi, birçok programlama dili araştırmacısı fonksiyonel programlamanın, emredici dillerle yapılan yordam temelli (procedural) programlamaya göre yazılım geliştirme açısından çok daha iyi bir yaklaşım olduğuna inanmaktadır.



Dillerin Etkileşimi

- Prolog'un yaygın kullanım alanlarından biri, bir tür akıllı veritabanı olarak kullanılmasıdır. Bu uygulama, Prolog dilini açıklamak için basit bir çerçeve sunar.
- Bir Prolog programının veritabanı iki tür ifadeden oluşur: **olgular (facts)** ve **kurallar (rules)**. Aşağıdakiler olgu ifadelerine örnektir:

```
mother(joanne, jake).
father(vern, joanne).
```

- Bu ifadeler, joanne'in jake'in annesi olduğunu ve vern'in joanne'in babası olduğunu belirtir.
- Bir kural ifadesine örnek:

```
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
```

- Bu ifade, belirli X, Y ve Z değerleri için, X'in Y'nin ebeveyni ve Y'nin de Z'nin ebeveyni olması durumunda, X'in Z'nin büyükanne/büyükbabası olduğu sonucunun çıkarılabileceğini belirtir.
- Prolog veritabanı, **hedef ifadeleri (goal statements)** ile etkileşimli olarak sorgulanabilir. Örneğin:

```
father(bob, darcie).
```

- Bu sorgu, bob'un darcie'nin babası olup olmadığını sorar. Böyle bir sorgu Prolog sistemine sunulduğunda, sistem çözümleme (resolution) sürecini kullanarak ifadenin doğruluğunu belirlemeye çalışır. Eğer hedefin doğru olduğu sonucuna varırsa "true", kanıtlayamazsa "false" çıktısını verir.

Dillerin Etkileşimi

- Java, C++ temel alınarak tasarlanmış; ancak daha küçük, daha basit ve daha güvenilir olacak şekilde geliştirilmiştir. Her iki dilde de sınıflar ve ilkel (primitive) türler vardır. Java'da diziler önceden tanımlı bir sınıfın örnekleridir ve indis sınır kontrolü otomatik yapılır; C++'ta ise bu yerleşik değildir.
- Java'da pointer yoktur; bunun yerine sınıf örneklerini işaret eden referans türleri kullanılır. Tüm nesneler heap üzerinde oluşturulur ve referanslar gerektiğinde otomatik olarak çözümlenir. Java'da boolean adlı ilkel bir mantıksal tür vardır ve C/C++'tan farklı olarak aritmetik ifadeler kontrol ifadelerinde kullanılamaz.
- Java'da bağımsız alt programlar yoktur; tüm fonksiyonlar sınıflar içinde tanımlı metotlardır. Bu nedenle Java yalnızca nesne yönelimli programlamayı desteklerken, C++ hem yordam temelli hem nesne yönelimli programlamayı destekler. Ayrıca C++ çoklu kalıtımı doğrudan desteklerken, Java yalnızca tekli kalıtımı destekler; çoklu kalıtımın bazı avantajları arayüzler (interface) ile sağlanır.
- Java'da struct ve union yapıları yoktur. Eşzamanlılık (concurrency) ise synchronized anahtar sözcüğü ile sağlanır; bu mekanizma kilit (lock) oluşturarak karşılıklı dışlamayı garanti eder. Java'da eşzamanlı süreçler thread olarak adlandırılır ve oluşturulmaları görece kolaydır.

Özet sorular

- Hangisi daha hızlı program yürütülmesini sağlar: bir derleyici mi yoksa saf bir yorumlayıcı mı?
- Aşırı ortogonallik neden sorunlara yol açar?
- Tür denetimi nedir? Nerede gerçekleşir?
- Bilimsel uygulamalar için hangi programlama dili ilk kez başarıyla kullanılmıştır?
- 1990'dan önce yapay zekâ uygulamalarının çoğu hangi programlama dilinde geliştirilmiştir?
- Bir programlama dilinin genel sadeliği, onun okunabilirliğini nasıl etkiler?
- Bazı dillerde gerekli olmamalarına rağmen, basit değişkenler için tür bildirim ifadeleri bir dilin okunabilirliğini nasıl etkiler?

Halting Problem

- Halting problemi diyorki bir programın sonlanıp sonlanmayacağına karar verebilen başka bir program yazılamaz. Derleyiciler programın çalışıp sonlanacağına bu programı çalıştırmadan karar veremezler. Bu durumda program çalıştırılırsa ve sonlanmaz ise kendisi ile çelişen bir kararsızlık durumu oluşacaktır.

Statik ve Dinamik Blok

- Blok nedir örnekler
- Statik blok işlem sonucu, dinamik blok işlem sonucu